Week 3 Group Practice Assignment

Smart Cold-Chain Delivery Monitoring Device
Program: coldchain_monitor.py | Device ID: coldbox_07

Course:		**Date:**	
Group:		**Members:**	

Part 1 - Program Output (Tasks 1 to 12)

Run **coldchain_monitor.py** and paste the screenshot of each task's output into the matching box below. Each box in the program output is marked with a banner such as *>>> SCREENSHOT FOR TASK 4 HERE*.

Task 1 - Modify the Virtual Sensors

Show temperature (0-12 C), humidity (20-85 %) and battery (10-100 %) with units.

Paste Task 1 screenshot here

Task 2 - Add Device Information

Show the device ID (coldbox_XX) and the current timestamp.

Paste Task 2 screenshot here

Task 3 - Validate the Sensor Measurements

Show VALID / INVALID for each measurement, including the deliberately invalid test values.

Paste Task 3 screenshot here

Task 4 - Determine Operating Status

Show TOO_COLD / NORMAL / TOO_WARM, HUMIDITY_WARNING and LOW_BATTERY results.

Paste Task 4 screenshot here

Task 5 - Create a Structured IoT Record

Show the printed nested dictionary (device_id, timestamp, measurements, quality, status).

Paste Task 5 screenshot here

Task 6 - Access Data from the Nested Dictionary

Show at least three values read out of the nested dictionary, including a measurement and a status.

Paste Task 6 screenshot here

Task 7 - Serialize the Record to JSON

Show both the normal JSON string and the indented, human-readable JSON.

Paste Task 7 screenshot here

Task 8 - Check the Data Types

Show the type before serialization (dict) and after serialization (str).

Paste Task 8 screenshot here

Task 9 - Measure the JSON Payload Size

Show the UTF-8 payload size in bytes.

Paste Task 9 screenshot here

Task 10 - Create Compact JSON

Show the compact JSON, both payload sizes and the percentage size reduction.

Paste Task 10 screenshot here

Task 11 - Generate Repeated IoT Messages

Show the start of the loop with the numbered readings and their compact JSON payloads.

Paste Task 11 screenshot here

Task 12 - Count Warning Conditions

Show the final summary: total readings and each warning counter.

Paste Task 12 screenshot here

Part 2 - Group Discussion Questions

Each question is answered in approximately 2 to 4 sentences.

1. Why is a nested dictionary more suitable than storing all measurements as separate variables?

A nested dictionary keeps one reading together as a single object, so the device ID, timestamp, measurements, quality and status all travel as one unit instead of six or seven loose variables. It also groups related items under meaningful names, so record["status"]["temperature"] says exactly what the value means. With separate variables we would have to pass every one of them into every function and rename them all if a new sensor were added, and they could not be serialized to JSON in one step.

2. What is the difference between data validation and operating status?

Validation asks whether the value is physically possible - a temperature of 85 C or a humidity of 120 % means the sensor is broken or the reading is corrupted, so the result is VALID or INVALID. Operating status asks whether a valid value is acceptable for the cargo - 11 C is a perfectly believable reading, but it is TOO_WARM for medicine that must stay between 2 and 8 C. Validation checks the sensor; status checks the shipment.

3. What is the difference between a Python dictionary and JSON?

A Python dictionary is an object that lives in memory inside a running program, and only Python can use it directly. JSON is a text format - after json.dumps() the data is just a string of characters that any language or system can read. The program output shows this: the type is <class 'dict'> before serialization and <class 'str'> after. The dictionary is the working form, JSON is the transport and storage form.

4. Why is serialization useful when IoT data must be stored or transmitted?

Networks and files move bytes, not Python objects, so the record has to be converted into a defined text format before it can be sent or saved. JSON is language-independent, so a device written in Python can send data to a server written in Java or JavaScript and it will still be understood. It also keeps the nested structure intact, so the receiver gets the measurements, quality and status groups exactly as the device built them, and can reverse the process with json.loads().

5. Why is compact JSON smaller even though the sensor measurements are unchanged?

The measurements are identical; only the formatting is removed. Indented JSON adds newline characters and many spaces so that humans can read it, and every one of those characters is a byte. Compact JSON drops the indentation and the spaces after the ':' and ',' separators. In our run the payload dropped from 418 bytes to 282 bytes - about a 32 % reduction - without losing a single value.

6. Why might reducing payload size be useful for an IoT device?

Smaller messages mean less radio transmission time, and the radio is usually the most power-hungry part of a battery-powered device - a cold-chain box has to survive a whole delivery on its battery. Many devices also use cellular or LPWAN connections where data is billed per megabyte or limited per message. Over 100 readings our saving is already about 13 KB, and a real box transmitting every few minutes for days would save far more bandwidth, power and cost.

Note on the number of records: Task 11 asks for 100 IoT records, while the "Required Program Flow" section lists 10. The program generates 100 as stated in the task. The value is set by the TOTAL_READINGS constant at the top of coldchain_monitor.py and can be changed in one line.